

RNN Outputs with Keras — Updated Notes

A practical guide to many-to-one, many-to-many, and selected hidden-state outputs

Core mental model: Every RNN time step creates a hidden state. The architecture decides which hidden states are consumed to make predictions.

1. RNN creates a hidden state at every time step

For a sequence $x_1 \dots x_8$, the RNN creates $h_1 \dots h_8$. If we use `return_sequences=True`, Keras gives us the entire sequence of hidden states rather than only h_8 .

```
states = LSTM(64, return_sequences=True)(inputs)

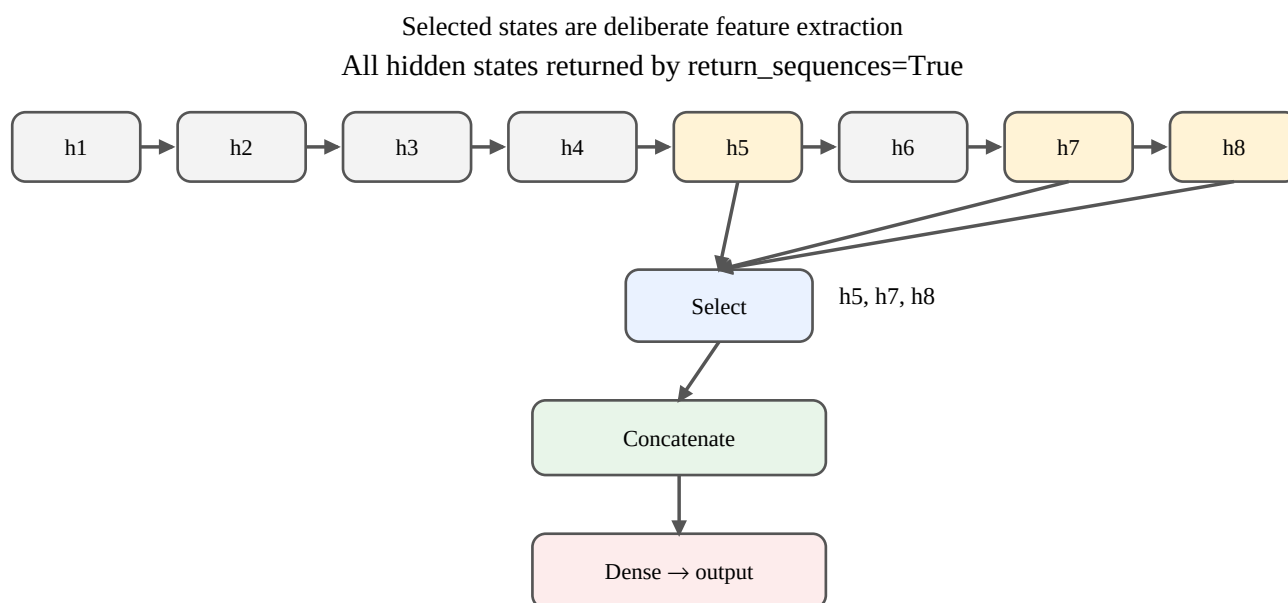
# Conceptual shape:
# states.shape = (batch_size, 8, 64)

# 8 = number of time steps
# 64 = hidden-state size
```

Think of **states** as a table: each row/time position has its own 64-dimensional representation.

2. Selected hidden states — the important idea

Suppose we only want **h_5 , h_7 and h_8** . We first ask the LSTM for all states, then extract those three positions. We are not asking the RNN to stop at h_5 or h_7 ; we are selecting their representations after the RNN has processed the sequence.



Visual logic:

```
x1 → h1
x2 → h2
x3 → h3
x4 → h4
x5 → h5 ■■
x6 → h6 ■
x7 → h7 ■■■ → concatenate → Dense → prediction
x8 → h8 ■■
```

The other states still exist. We simply do not feed h_1 , h_2 , h_3 , h_4 or h_6 into this particular output head.

3. Better Keras implementation

For teaching and real models, the Functional API makes the selection explicit. Here we use **Lambda** layers so the tensor slicing is visible inside the Keras model graph.

```
import tensorflow as tf
from tensorflow.keras import Model, Input
from tensorflow.keras.layers import LSTM, Lambda, Concatenate, Dense

# -----
# Input
# -----
# 8 time steps, 10 features at each time step
inputs = Input(shape=(8, 10), name="sequence")

# -----
```

```

# RNN
# -----
# Keep the hidden state from EVERY time step
states = LSTM(
    64,
    return_sequences=True,
    name="lstm"
)(inputs)

# states shape:
# (batch_size, 8, 64)

# -----
# Select h5, h7 and h8
# -----
h5 = Lambda(
    lambda x: x[:, 4, :],
    name="select_h5"
)(states)

h7 = Lambda(
    lambda x: x[:, 6, :],
    name="select_h7"
)(states)

h8 = Lambda(
    lambda x: x[:, 7, :],
    name="select_h8"
)(states)

# Each selected state:
# (batch_size, 64)

# -----
# Combine selected states
# -----
combined = Concatenate(
    name="selected_states"
)([h5, h7, h8])

# combined shape:
# (batch_size, 192)
# because 64 + 64 + 64 = 192

# -----
# Final prediction
# -----
output = Dense(
    1,
    name="regression_output"
)(combined)

model = Model(
    inputs=inputs,
    outputs=output
)

model.summary()

```

4. Why are the indices 4, 6 and 7?

Python uses zero-based indexing. Therefore the fifth hidden state is index 4, the seventh is index 6, and the eighth is index 7.

Human meaning	Python index	Selection
h1	0	states[:, 0, :]
h2	1	states[:, 1, :]
h5	4	states[:, 4, :]
h7	6	states[:, 6, :]
h8	7	states[:, 7, :]

5. Understand the tensor shapes

```
Input:
(batch, 8, 10)

↓ LSTM(64, return_sequences=True)

All states:
(batch, 8, 64)

↓ select h5, h7, h8

h5 = (batch, 64)
h7 = (batch, 64)
h8 = (batch, 64)

↓ Concatenate

combined = (batch, 192)

↓ Dense(1)

output = (batch, 1)
```

This is the most useful implementation detail: selecting states changes the time dimension into a chosen set of feature vectors. Concatenating three 64-dimensional states creates a 192-dimensional feature vector for each sample.

6. A realistic example

Imagine a sequence of 8 measurements and a business rule that positions 5, 7 and 8 are especially important for the final prediction. The model can explicitly extract those states and use them as features.

```
# Example:
# x1 x2 x3 x4 x5 x6 x7 x8
#      ↑   ↑   ↑
#      h5  h7 h8

selected = Concatenate()([h5, h7, h8])
prediction = Dense(1)(selected)
```

In practice, you would normally have a reason for selecting positions. You could also let the model learn which states matter using attention instead of hard-coding the positions.

7. Compare the three designs

Design	What you use	Typical output	Example
Last state	h8 only	One or several values	Sequence → regression

All states	h1...h8	One prediction per step	NER
Selected states	h5,h7,h8	One or several values	Custom feature extraction

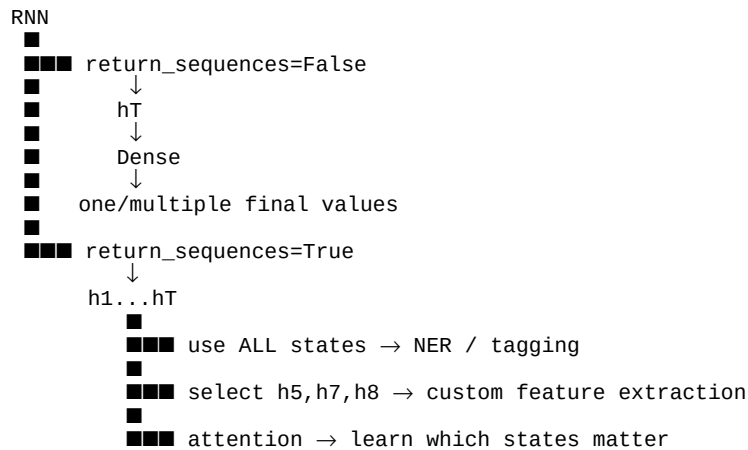
8. Important caveat: selected states are not independent

An RNN carries information forward. Therefore h_8 was computed from h_7 , and h_7 already contains information carried from earlier states. Selecting h_5 , h_7 and h_8 does **not** mean we have three completely independent summaries.

$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4 \rightarrow h_5 \rightarrow h_6 \rightarrow h_7 \rightarrow h_8$
 ↑ ↑ ↑
 select select select

The selection is an architectural choice: we are saying that these representations are useful for the downstream task.

9. The bigger picture



Final takeaway: Every time step creates a hidden representation. Keras lets you expose all of them, select specific ones, combine them, and then attach whatever prediction head your problem needs.